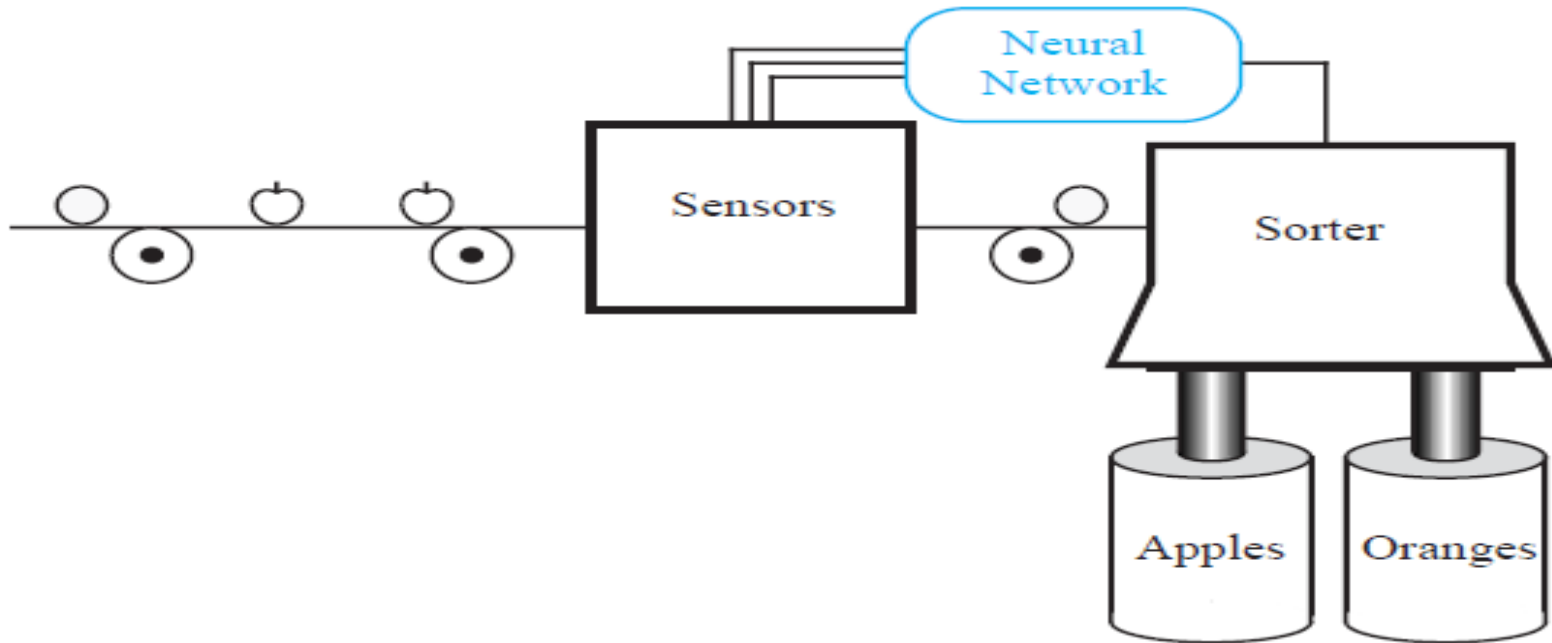
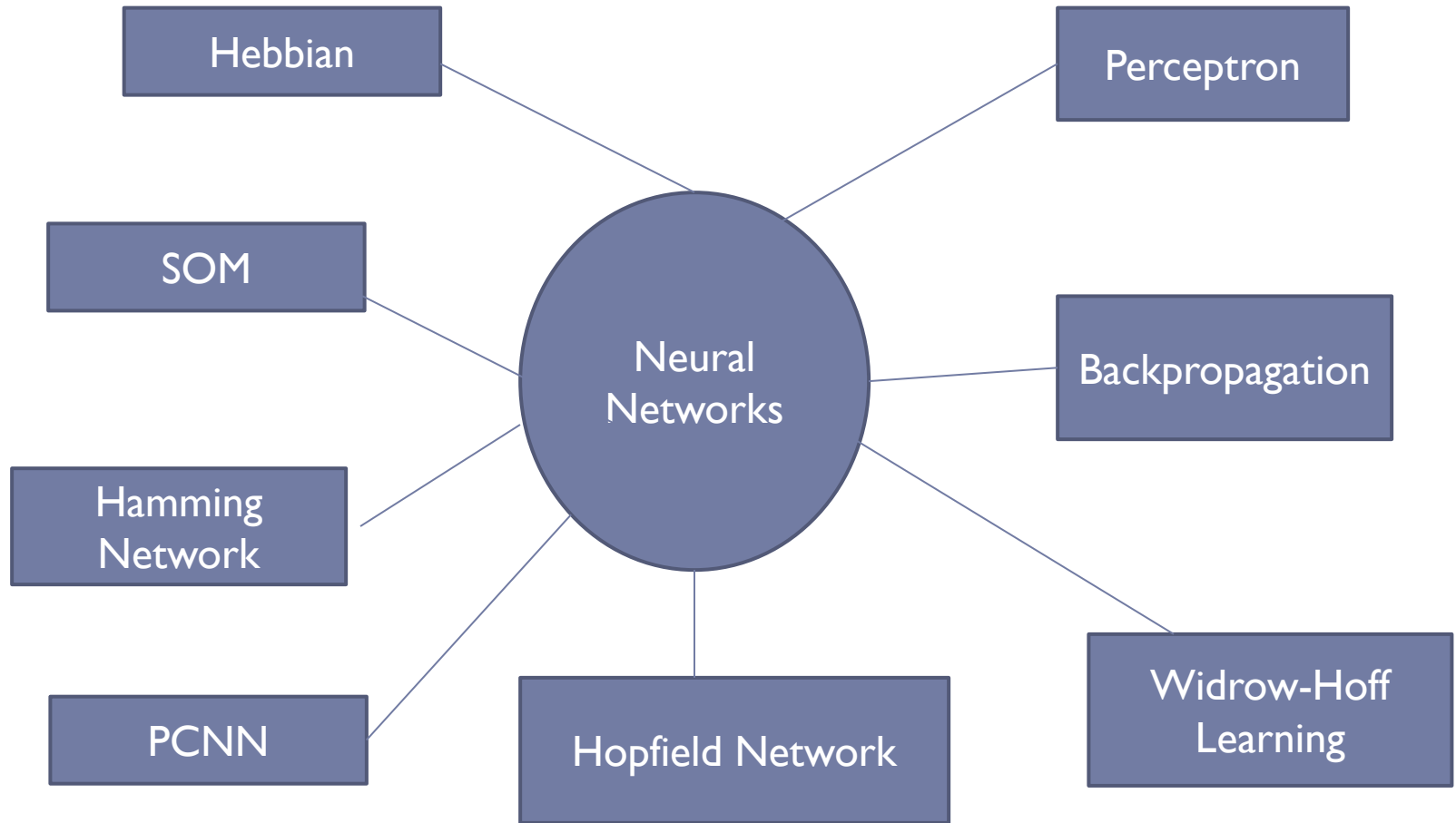


Neural Networks



Main Topics



Case study



Biological neuron

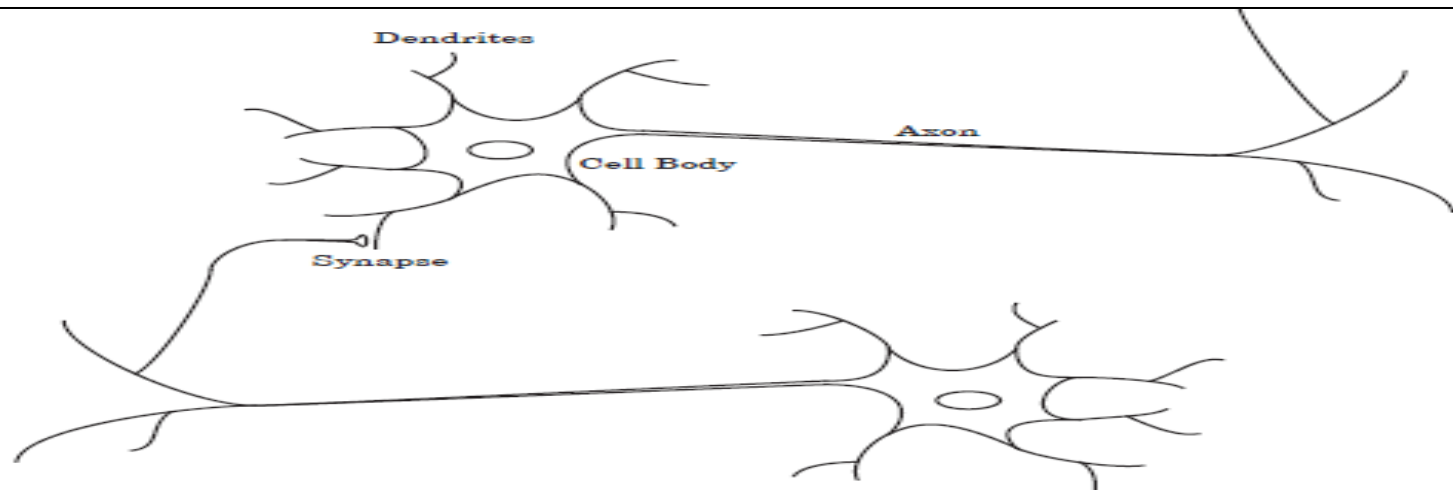


Figure 1.1 Schematic Drawing of Biological Neurons

The brain consists of a large number of highly connected elements (approximately 104 connections per element) called neurons.

neurons have three principal components: the dendrites, the cell body and the axon.



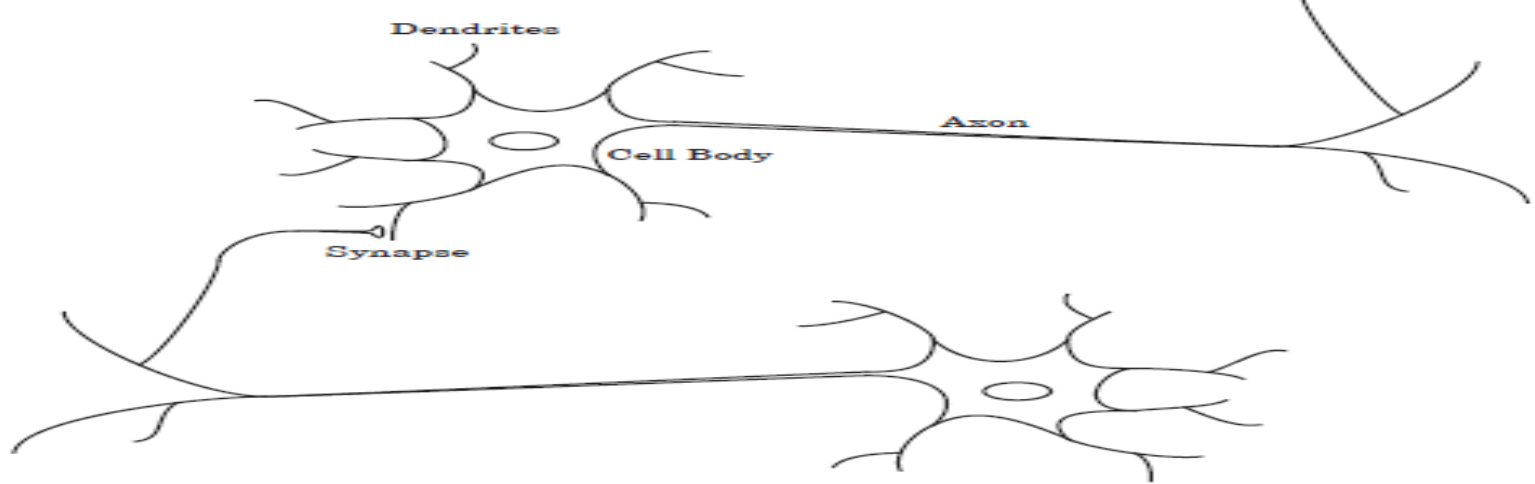


Figure 1.1 Schematic Drawing of Biological Neurons

- ▶ The **dendrites** are tree-like receptive networks of nerve fibers that carry electrical signals into the cell body.
- ▶ The **cell body** effectively sums and thresholds these incoming signals.
- ▶ The **axon** is a single long fiber that carries the signal from the cell body out to other neurons.
- ▶ The point of contact between an axon of one cell and a dendrite of another cell is called a synapse.

Neuron Model

Single-Input Neuron

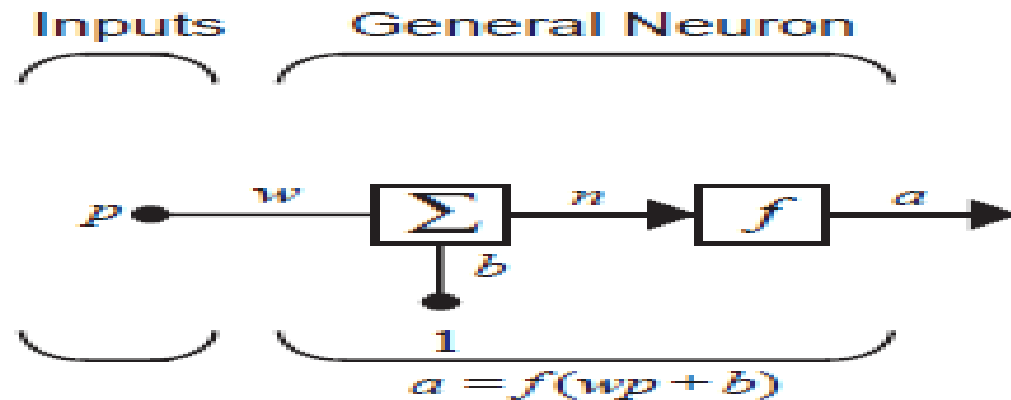


Figure 2.1 Single-Input Neuron

The neuron output is calculated as

$$a = f(wp + b).$$

If, for instance, $w = 3$, $p = 2$ and $b = -1.5$, then

$$a = f(3(2) - 1.5) = f(4.5)$$

Transfer Functions

- ▶ The transfer function may be a linear or a nonlinear function

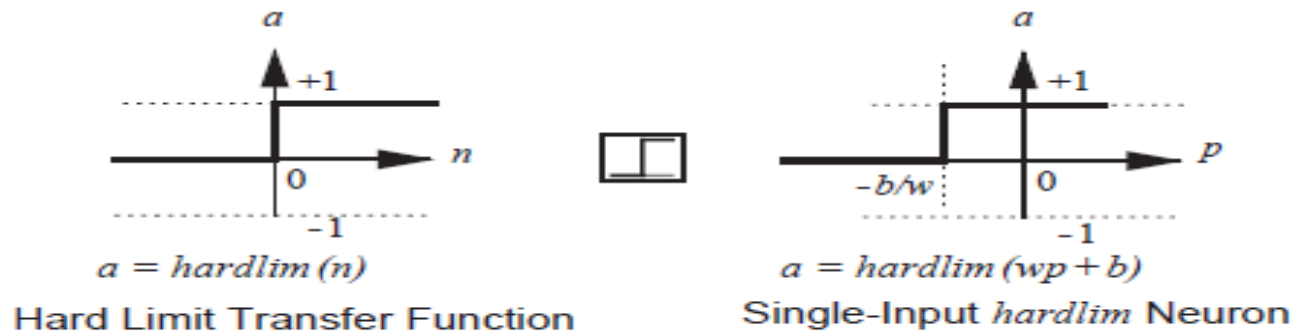


Figure 2.2 Hard Limit Transfer Function

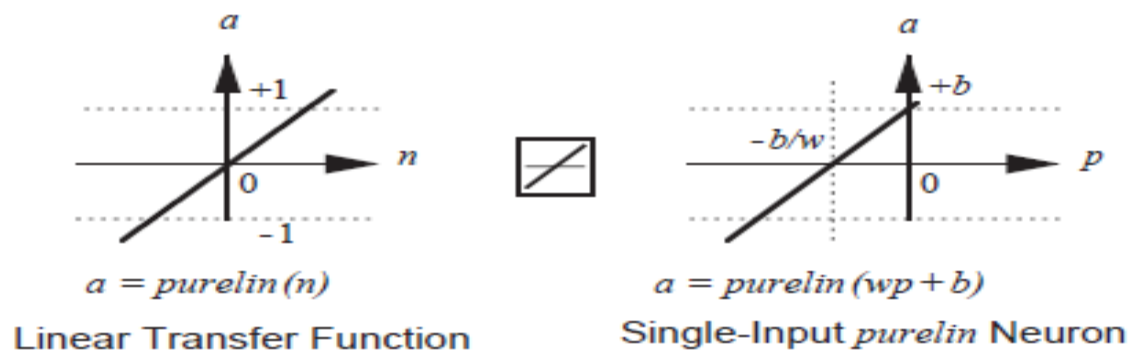
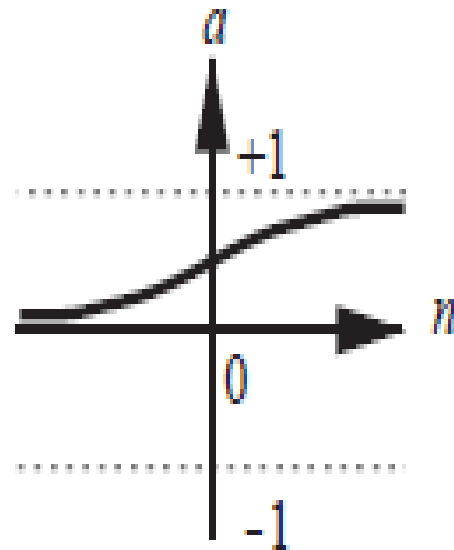
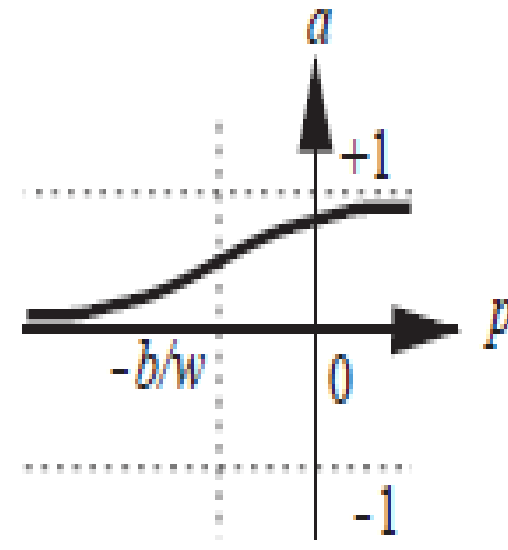
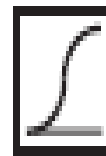


Figure 2.3 Linear Transfer Function



$$a = \text{logsig}(n)$$

Log-Sigmoid Transfer Function



$$a = \text{logsig}(wp + b)$$

Single-Input *logsig* Neuron

Figure 2.4 Log-Sigmoid Transfer Function

Name	Input/Output Relation	Icon	MATLAB Function
Hard Limit	$a = 0 \quad n < 0$ $a = 1 \quad n \geq 0$		hardlim
Symmetrical Hard Limit	$a = -1 \quad n < 0$ $a = +1 \quad n \geq 0$		hardlims
Linear	$a = n$		purelin
Saturating Linear	$a = 0 \quad n < 0$ $a = n \quad 0 \leq n \leq 1$ $a = 1 \quad n > 1$		satlin
Symmetric Saturating Linear	$a = -1 \quad n < -1$ $a = n \quad -1 \leq n \leq 1$ $a = 1 \quad n > 1$		satlins
Log-Sigmoid	$a = \frac{1}{1 + e^{-n}}$		logsig
Hyperbolic Tangent Sigmoid	$a = \frac{e^n - e^{-n}}{e^n + e^{-n}}$		tansig
Positive Linear	$a = 0 \quad n < 0$ $a = n \quad 0 \leq n$		poslin
Competitive	$a = 1 \quad \text{neuron with max } n$ $a = 0 \quad \text{all other neurons}$		compet

Table 2.1 Transfer Functions

2. Multiple-Input Neuron

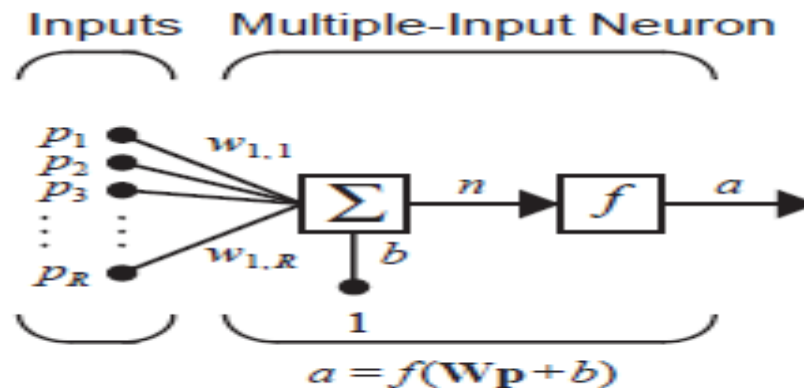


Figure 2.5 Multiple-Input Neuron

The neuron has a bias b , which is summed with the weighted inputs to form the net input n :

$$n = w_{1,1}p_1 + w_{1,2}p_2 + \dots + w_{1,R}p_R + b. \quad (2.3)$$

This expression can be written in matrix form:

$$n = W\mathbf{p} + b, \quad (2.4)$$

where the matrix W for the single neuron case has only one row.

Now the neuron output can be written as

$$a = f(W\mathbf{p} + b). \quad (2.5)$$

Abbreviation notation

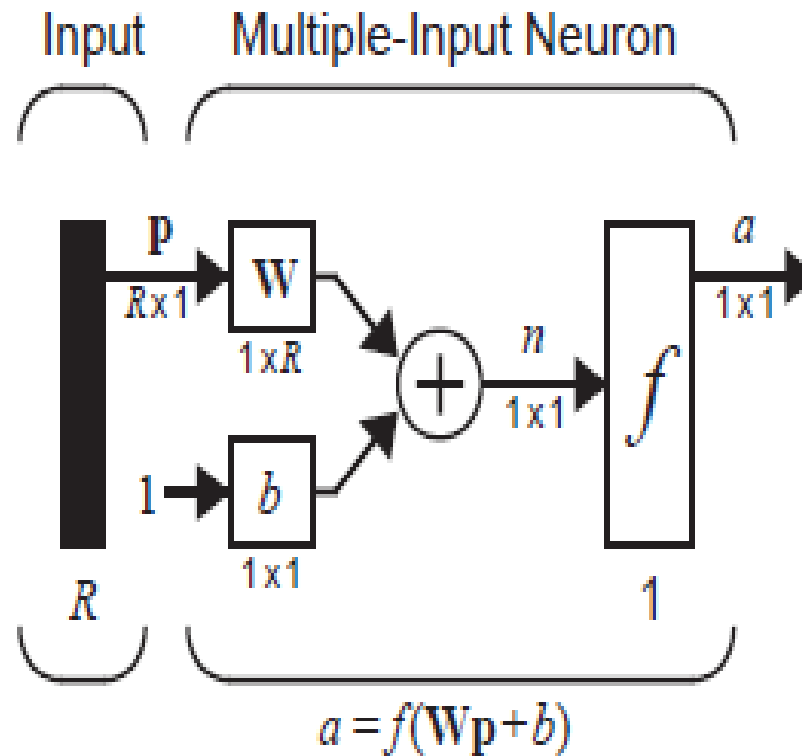


Figure 2.6 Neuron with R Inputs, Abbreviated Notation

Network Architectures

A Layer of Neurons

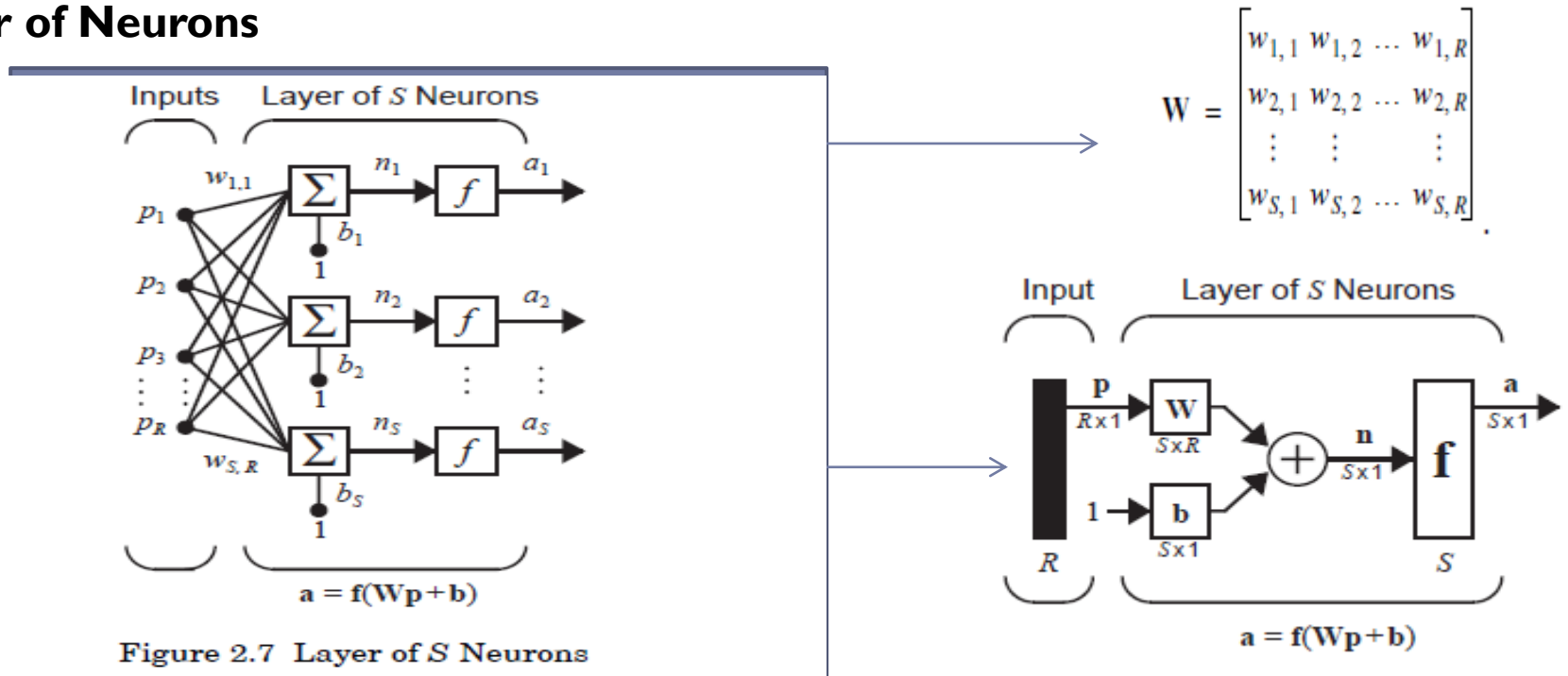


Figure 2.7 Layer of S Neurons

The layer includes the weight matrix, the summers, the bias vector, the transfer function boxes and the output vector. Each element of the input vector is connected to each neuron through the weight matrix. Each neuron has a bias, a transfer function and an output. Taken together, the outputs form the output vector.

Multiple Layers of Neurons

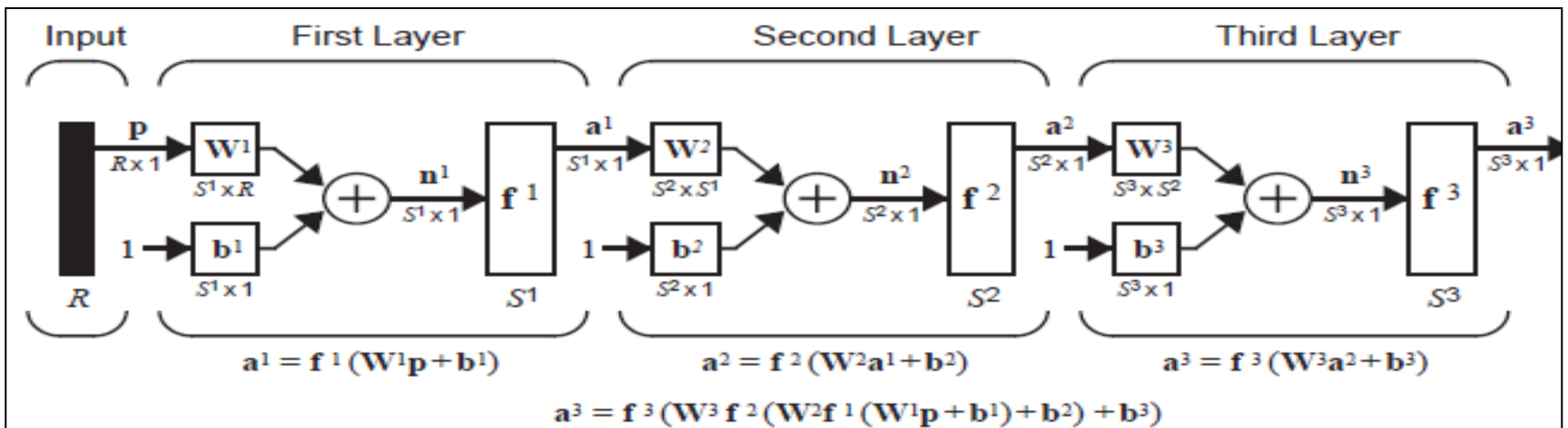
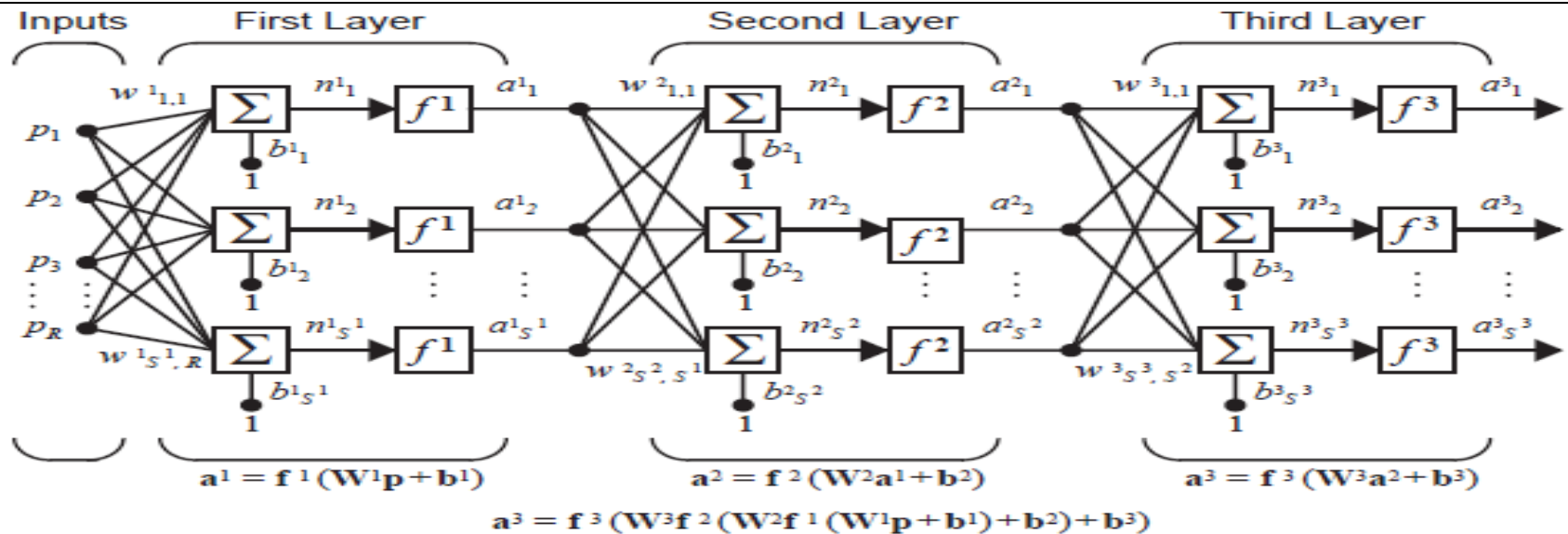


Figure 2.10 Three-Layer Network, Abbreviated Notation

problem 1

- ▶ **P2.1** The input to a single-input neuron is 2.0, its weight is 2.3 and its bias is -3.

i. What is the net input to the transfer function?

ii. What is the neuron output?

soln

- ▶ **i.** The net input is given by: $n = wp + b = (2.3)(2) + (-3) = 1.6$
- ▶ **ii.** The output cannot be determined because the transfer function is not specified.



problem2

- ▶ **P2.2 What is the output of the neuron of P2.1 if it has the following transfer functions?**
- ▶ **i. Hard limit**
- ▶ **ii. Linear**
- ▶ **iii. Log-sigmoid**

- ▶ **i. For the hard limit transfer function:**

$$a = \text{hardlim}(1.6) = 1.0$$

- ▶ **ii. For the linear transfer function:**

$$a = \text{purelin}(1.6) = 1.6$$

- ▶ **iii. For the log-sigmoid transfer function**

$$a = \text{logsig}(1.6) = \frac{1}{1 + e^{-1.6}} = 0.8320$$

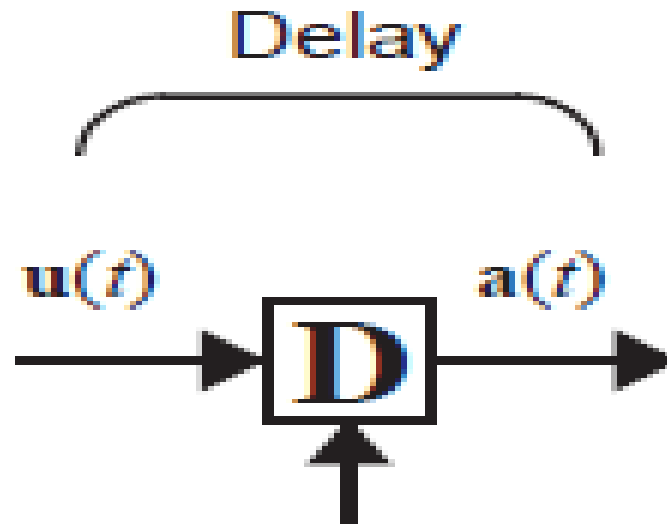


Assignment

- ▶ Build database of iris or ear or face or fingerprint or DNA
- ▶ Apply perceptron to 10 images



Delay Block



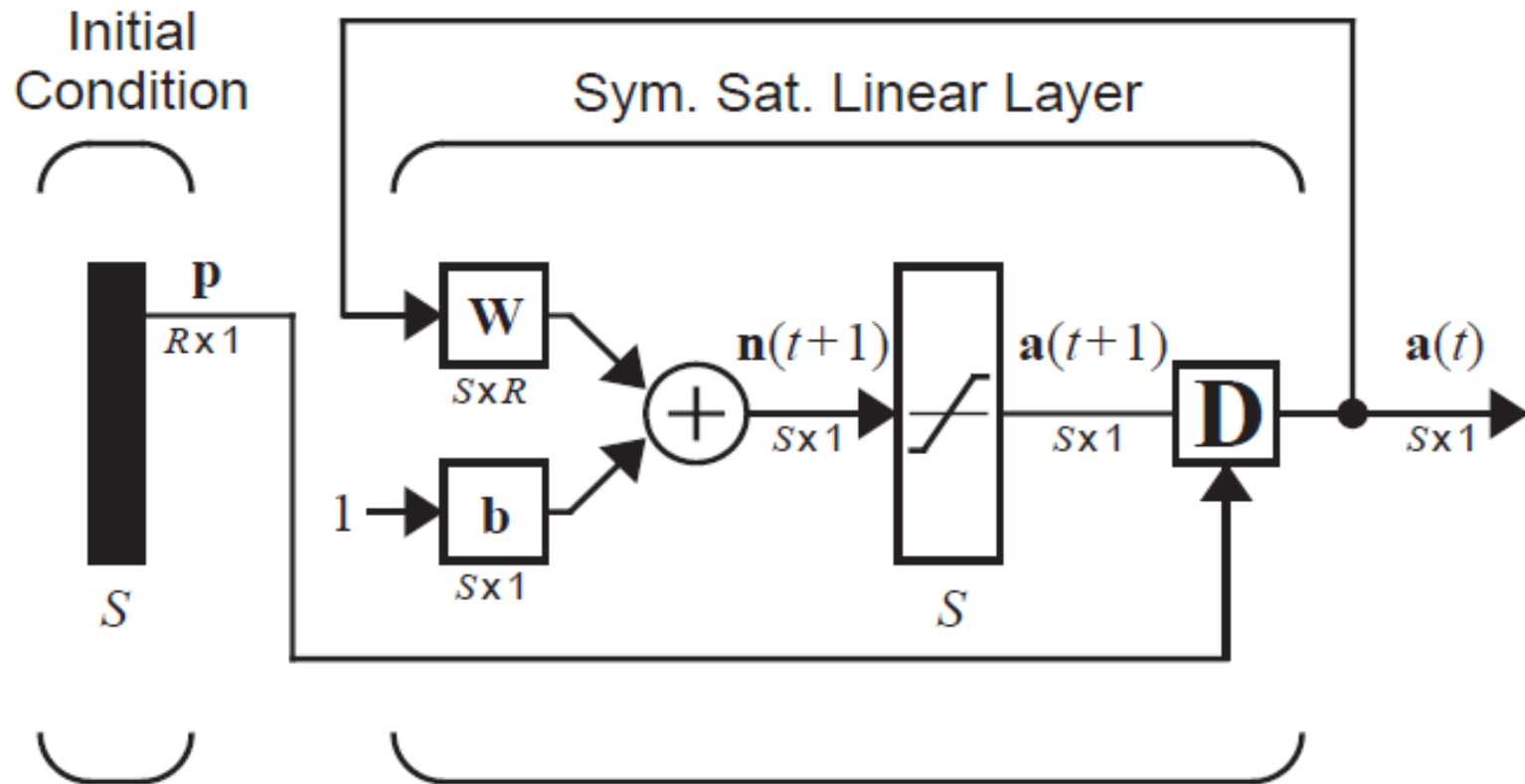
The delay output $a(t)$ is computed from its input $u(t)$ according to

$$a(t) = u(t - 1).$$

$a(t) = u(t - 1)$

Figure 2.11 Delay Block

Recurrent Network



$$\mathbf{a}(0) = \mathbf{p} \quad \mathbf{a}(t+1) = \text{satlin}(\mathbf{W}\mathbf{a}(t) + \mathbf{b})$$

$$\mathbf{a}(1) = \text{satlins}(\mathbf{W}\mathbf{a}(0) + \mathbf{b}) = \text{satlins}(\mathbf{W}\mathbf{p} + \mathbf{b})$$

$$\mathbf{a}(2) = \text{satlins}(\mathbf{W}\mathbf{a}(1) + \mathbf{b})$$

Problem



P2.4 A single-layer neural network is to have six inputs and two outputs. The outputs are to be limited to and continuous over the range 0 to 1. What can you tell about the network architecture? Specifically:

- i. How many neurons are required?**
- ii. What are the dimensions of the weight matrix?**
- iii. What kind of transfer functions could be used?**
- iv. Is a bias required?**

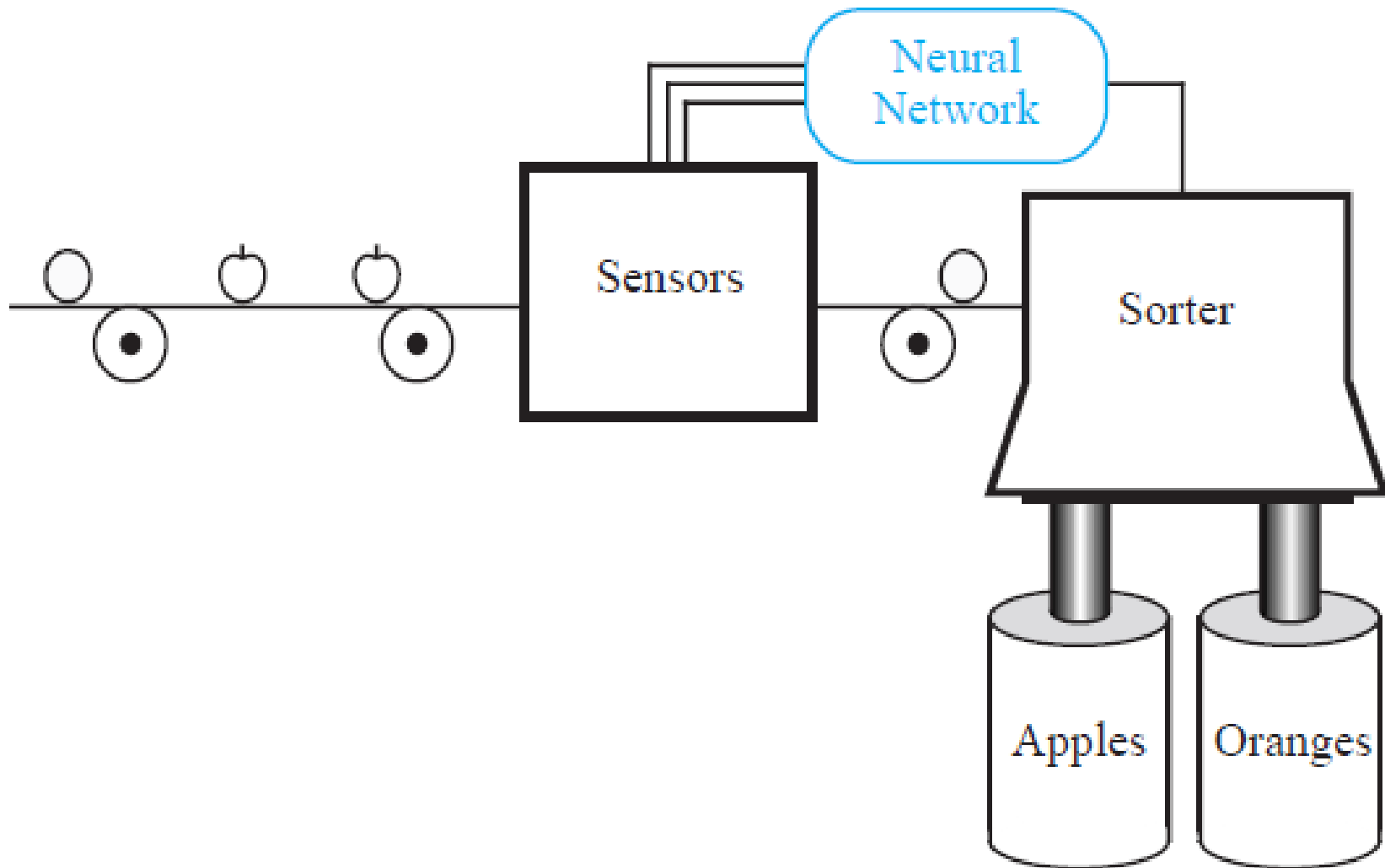
The problem specifications allow you to say the following about the network.

- i. Two neurons, one for each output, are required.**
- ii. The weight matrix has two rows corresponding to the two neurons and six columns corresponding to the six inputs. (The product Wp is a two-element vector.)**
- iii. Of the transfer functions we have discussed, the *logsig* transfer function would be most appropriate.**
- iv. Not enough information is given to determine if a bias is required.**



An illustrative example





Prototype Vectors

$$\mathbf{p} = \begin{bmatrix} \textit{shape} \\ \textit{texture} \\ \textit{weight} \end{bmatrix}.$$

Therefore, a prototype orange would be represented by

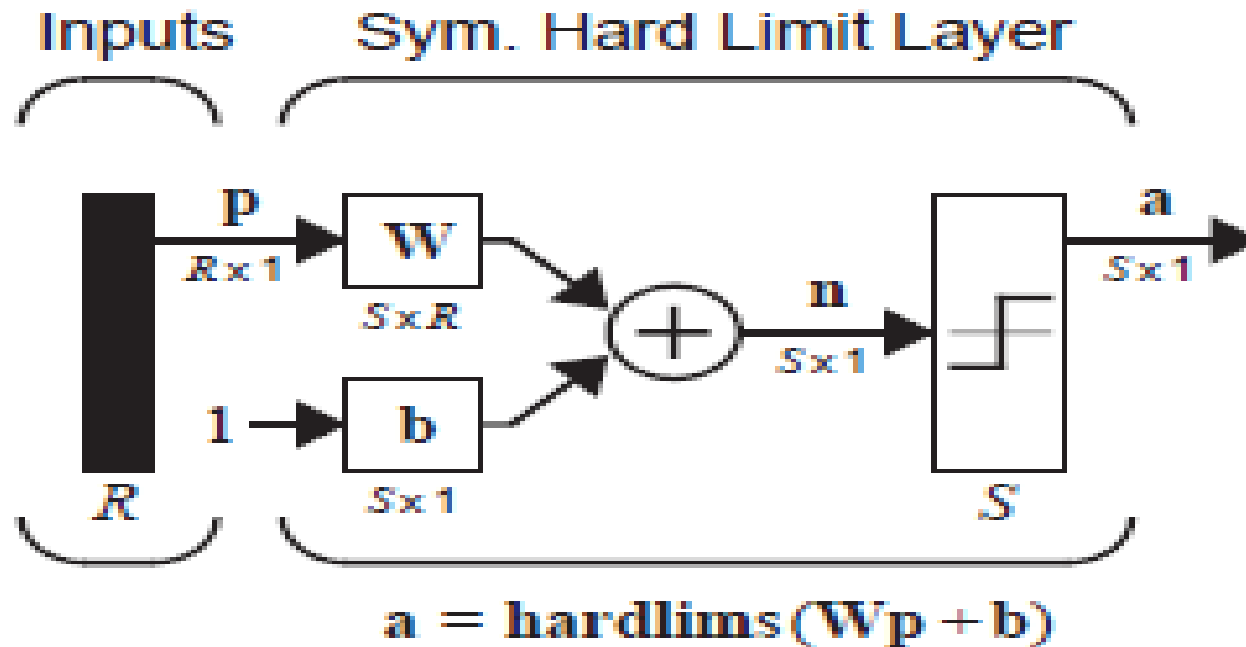
$$\mathbf{p}_1 = \begin{bmatrix} 1 \\ -1 \\ -1 \end{bmatrix},$$

and a prototype apple would be represented by

$$\mathbf{p}_2 = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix}.$$



Perceptron



Single layer Perceptron

Feedforward Network
Linear Decision Boundary
One Neuron for each decision

Two input case

$$a = \text{hardlims}(n) = \text{hardlims}\left(\begin{bmatrix} -1 & 1 \end{bmatrix} \mathbf{p} + b\right) .$$

$$n = \begin{bmatrix} -1 & 1 \end{bmatrix} \mathbf{p} - 1 = 0 .$$

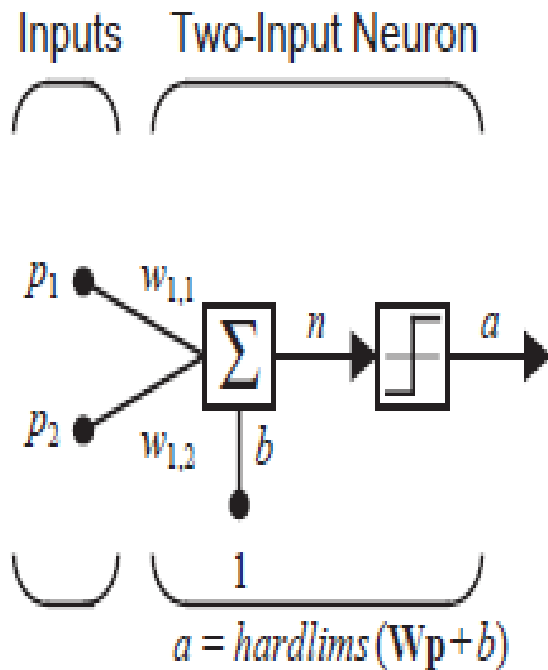


Figure 3.2 Two-Input/Single-Neuron Perceptron

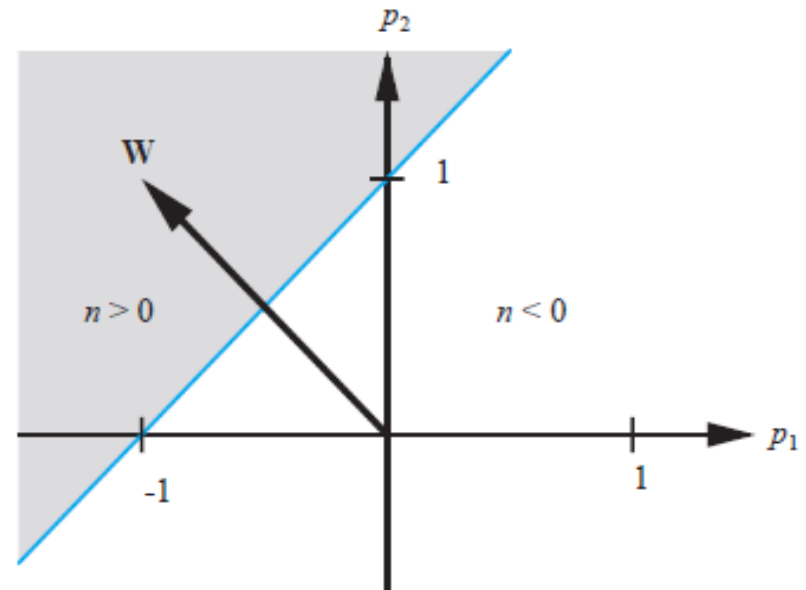


Figure 3.3 Perceptron Decision Boundary

Pattern recognition example

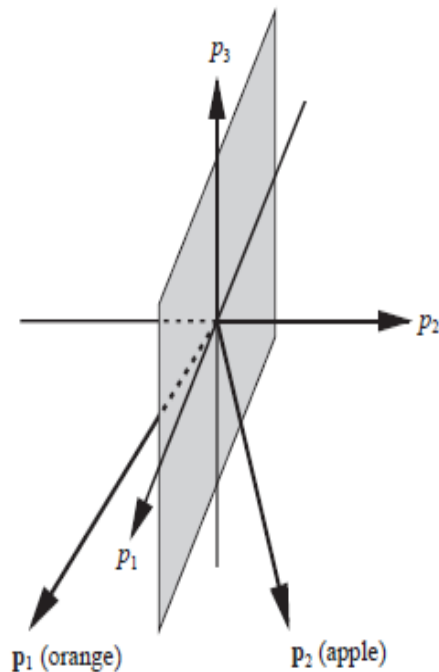


Figure 3.4 Prototype Vectors

The p_1, p_3 plane, which will be our decision boundary, can be described by the equation

$$p_2 = 0, \quad (3.8)$$

or

$$\begin{bmatrix} 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} p_1 \\ p_2 \\ p_3 \end{bmatrix} + 0 = 0. \quad (3.9)$$

Therefore the weight matrix and bias will be

$$W = \begin{bmatrix} 0 & 1 & 0 \end{bmatrix}, b = 0. \quad (3.10)$$

The weight matrix is orthogonal to the decision boundary and points toward the region that contains the prototype pattern p_2 (*apple*) for which we want the perceptron to produce an output of 1. The bias is 0 because the decision boundary passes through the origin.

Testing Network

Orange:

$$a = \text{hardlims} \left(\begin{bmatrix} 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \\ -1 \end{bmatrix} + 0 \right) = -1(\text{orange}) ,$$

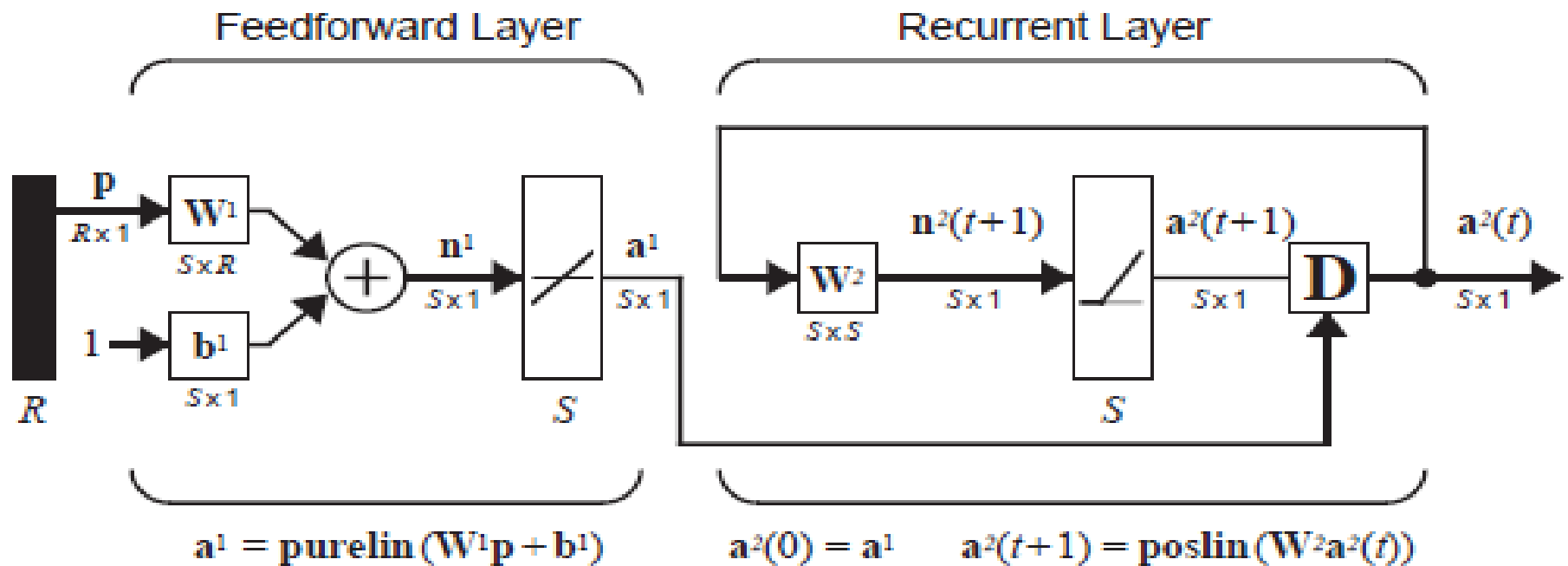
Apple:

$$a = \text{hardlims} \left(\begin{bmatrix} 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix} + 0 \right) = 1(\text{apple}) .$$

But what happens if we put a not-so-perfect orange into the classifier?

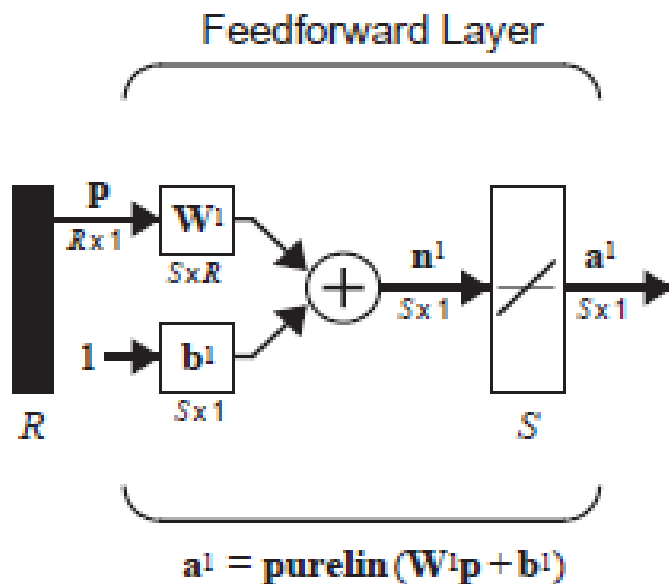


Hamming Network



Feedforward Layer

For Banana/Apple Recognition



$$S = 2$$

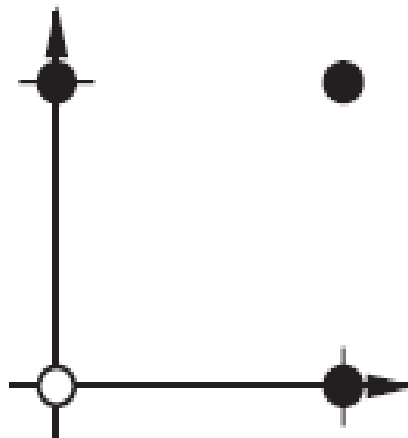
$$\mathbf{W}^1 = \begin{bmatrix} \mathbf{p}_1^T \\ \mathbf{p}_2^T \end{bmatrix} = \begin{bmatrix} -1 & 1 & -1 \\ 1 & 1 & -1 \end{bmatrix}$$

$$\mathbf{b}^1 = \begin{bmatrix} R \\ R \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}$$

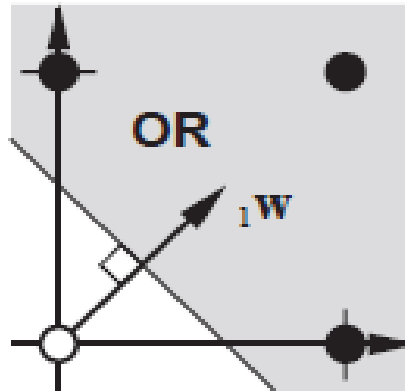
$$\mathbf{a}^1 = \mathbf{W}^1 \mathbf{p} + \mathbf{b}^1 = \begin{bmatrix} \mathbf{p}_1^T \\ \mathbf{p}_2^T \end{bmatrix} \mathbf{p} + \begin{bmatrix} 3 \\ 3 \end{bmatrix} = \begin{bmatrix} \mathbf{p}_1^T \mathbf{p} + 3 \\ \mathbf{p}_2^T \mathbf{p} + 3 \end{bmatrix}$$

Example

$$\left\{ \mathbf{p}_1 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, t_1 = 0 \right\} \quad \left\{ \mathbf{p}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, t_2 = 1 \right\} \quad \left\{ \mathbf{p}_3 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, t_3 = 1 \right\} \quad \left\{ \mathbf{p}_4 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, t_4 = 1 \right\}$$



OR Solution



Weight vector should be orthogonal to the decision boundary.

$${}_1\mathbf{w} = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$$

Pick a point on the decision boundary to find the bias.

$${}_1\mathbf{w}^T \mathbf{p} + b = \begin{bmatrix} 0.5 & 0.5 \end{bmatrix} \begin{bmatrix} 0 \\ 0.5 \end{bmatrix} + b = 0.25 + b = 0 \quad \Rightarrow \quad b = -0.25$$



Multiple-Neuron Perceptron

Each neuron will have its own decision boundary.

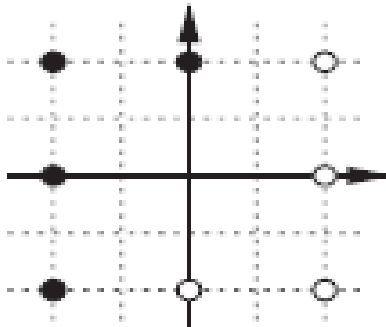
$${}_i\mathbf{w}^T \mathbf{p} + b_i = 0$$

A single neuron can classify input vectors
into two categories.

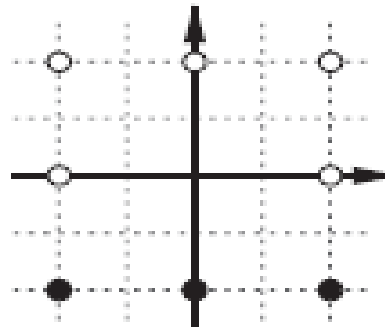
A multi-neuron perceptron can classify
input vectors into 2^S categories.



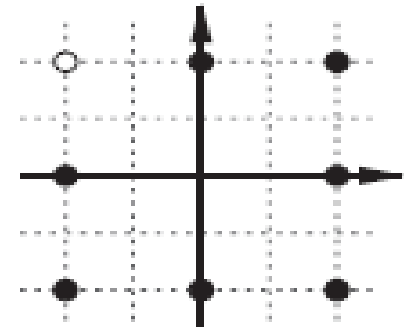
P4.1 Solve the three simple classification problems shown in Figure P4.1 by drawing a decision boundary. Find weight and bias values that result in single-neuron perceptrons with the chosen decision boundaries.



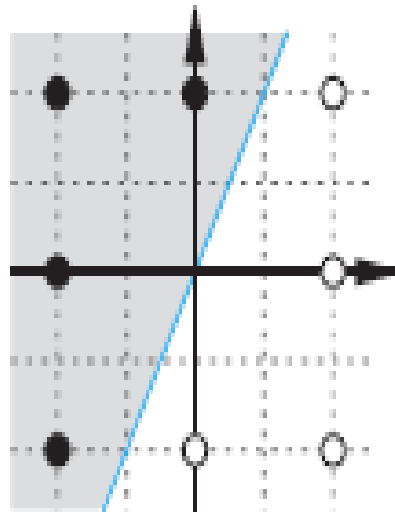
(a)



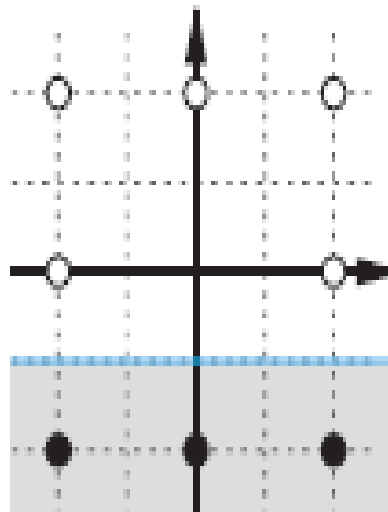
(b)



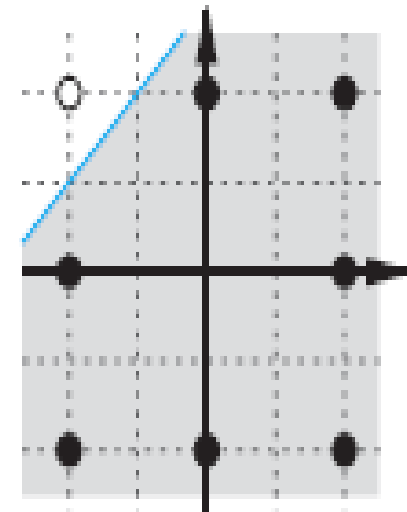
(c)



(a)

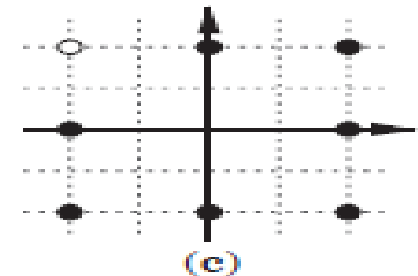
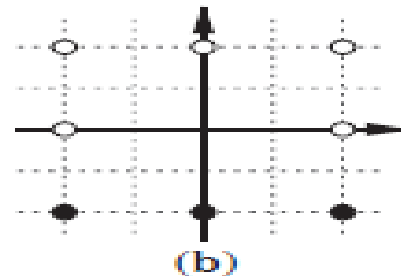
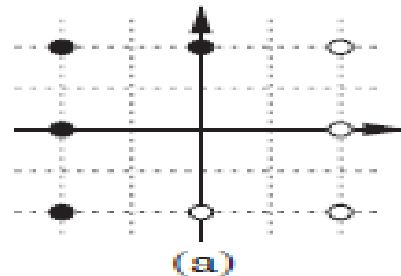


(b)

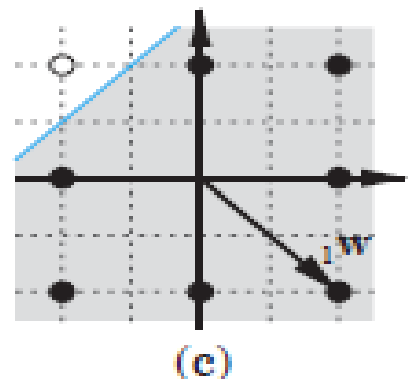
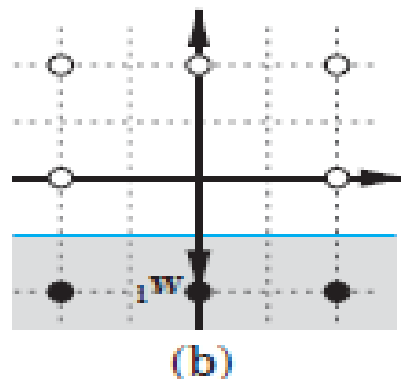
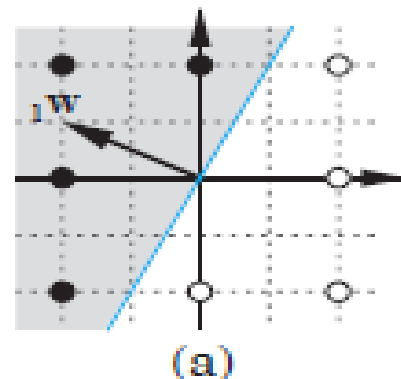


(c)

P4.1 Solve the three simple classification problems shown in Figure P4.1 by drawing a decision boundary. Find weight and bias values that result in single-neuron perceptrons with the chosen decision boundaries.



The next step is to find the weights and biases. The weight vectors must be orthogonal to the decision boundaries, and pointing in the direction of points to be classified as 1 (the dark points). The weight vectors can have any length we like.



Here is one set of choices for the weight vectors:

$$(a) \quad {}_1\mathbf{w}^T = \begin{bmatrix} -2 & 1 \end{bmatrix}, \quad (b) \quad {}_1\mathbf{w}^T = \begin{bmatrix} 0 & -2 \end{bmatrix}, \quad (c) \quad {}_1\mathbf{w}^T = \begin{bmatrix} 2 & -2 \end{bmatrix}.$$



Now we find the bias values for each perceptron by picking a point on the decision boundary and satisfying Eq. (4.15).

$$\begin{aligned} {}_1\mathbf{w}^T \mathbf{p} + b &= 0 \\ b &= -{}_1\mathbf{w}^T \mathbf{p} \end{aligned}$$

This gives us the following three biases:

$$\text{(a) } b = -\begin{bmatrix} -2 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 0, \text{ (b) } b = -\begin{bmatrix} 0 & -2 \end{bmatrix} \begin{bmatrix} 0 \\ -1 \end{bmatrix} = -2, \text{ (c) } b = -\begin{bmatrix} 2 & -2 \end{bmatrix} \begin{bmatrix} -2 \\ 1 \end{bmatrix} = 6$$

We can now check our solution against the original points. Here we test the first network on the input vector $\mathbf{p} = \begin{bmatrix} -2 & 2 \end{bmatrix}^T$.

$$\begin{aligned} a &= \text{hardlim}({}_1\mathbf{w}^T \mathbf{p} + b) \\ &= \text{hardlim}\left(\begin{bmatrix} -2 & 1 \end{bmatrix} \begin{bmatrix} -2 \\ 2 \end{bmatrix} + 0\right) \\ &= \text{hardlim}(6) \\ &= 1 \end{aligned}$$